

Deterministic Frame-based Testing

Maintaining determinism and extracting module-tests for
autonomous driving planning

 Yuqi Huai  Yuntianyi Chen  Ziwen Wan  Qi Alfred Chen  Joshua Garcia

ICSE 2026 Research Track Paper DOI 10.1145/3744916.3773252

The National Highway Traffic Safety Administration (NHTSA)

"Types of automated technologies, such as advanced driver assistance system technologies already in use on the roads and future automated driving systems at their mature state, have the potential to reduce crashes, prevent injuries, and save lives. In some circumstances, automated technologies may be able to detect the threat of a crash and act faster than drivers. These technologies could greatly support drivers and reduce human errors and the resulting crashes, injuries, and economic tolls."

In reality ...

The collage consists of three main elements:

- Left Article:** A news snippet from ABC 7 titled "Waymo to launch in Los Angeles, but not in San Francisco". It includes a photo of a Waymo robotaxi and mentions a crash in Los Angeles on March 11. The author is Rachel Urie.
- Middle Article:** A news snippet from NBC News titled "Driver hits pedestrian of self-driving car". It mentions a woman in critical condition and is dated October 3, 2023. The author is David Ingram.
- Right Article:** A full news article from ABC News titled "Federal officials investigate after Waymo self-driving vehicle strikes child near elementary school". It includes a video player showing a white Waymo vehicle on a street. The article states that the child sustained minor injuries and that federal officials are investigating. It also mentions that the incident occurred on January 23 in Santa Monica.

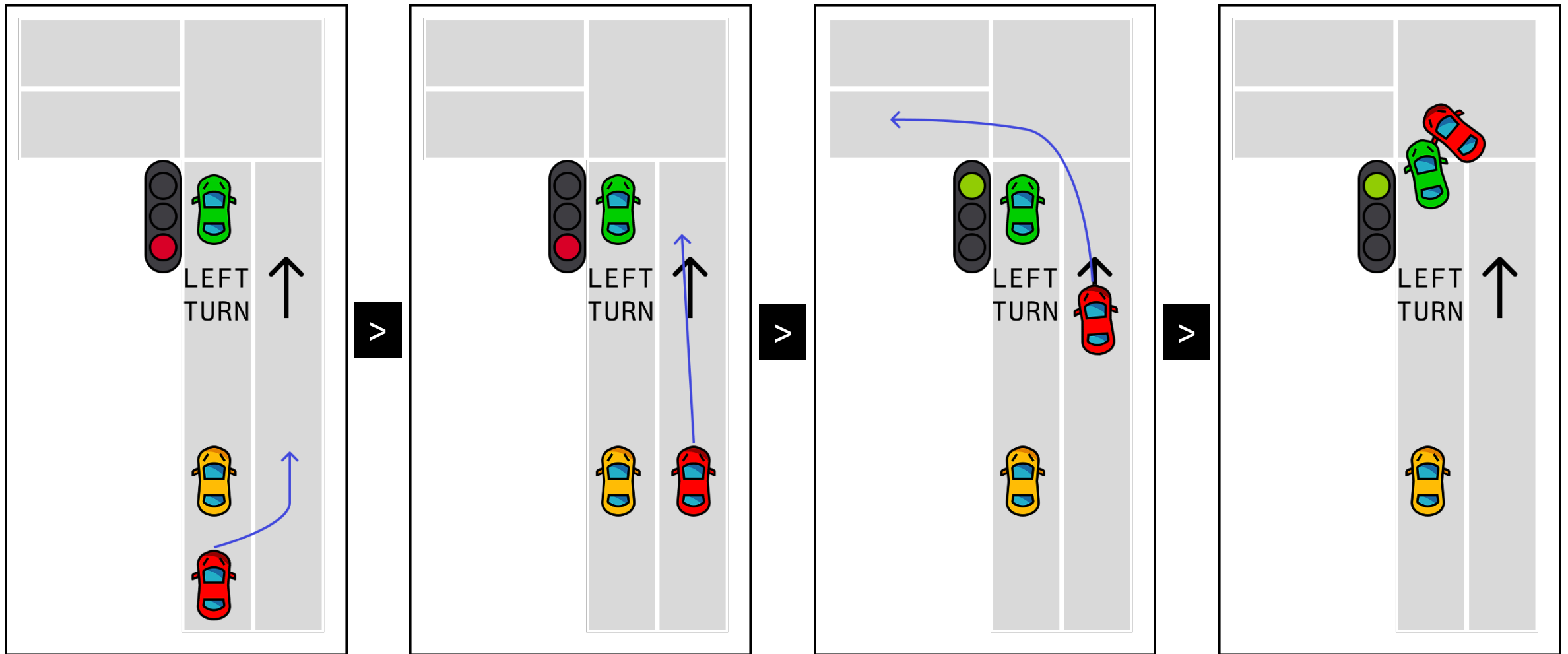


Bug Revealing Scenario!

20%
REPRODUCTION RATE

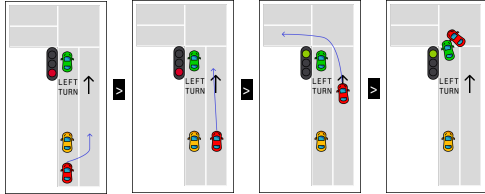
Failing Test Executions (4/20)

AV Cuts off **another vehicle**



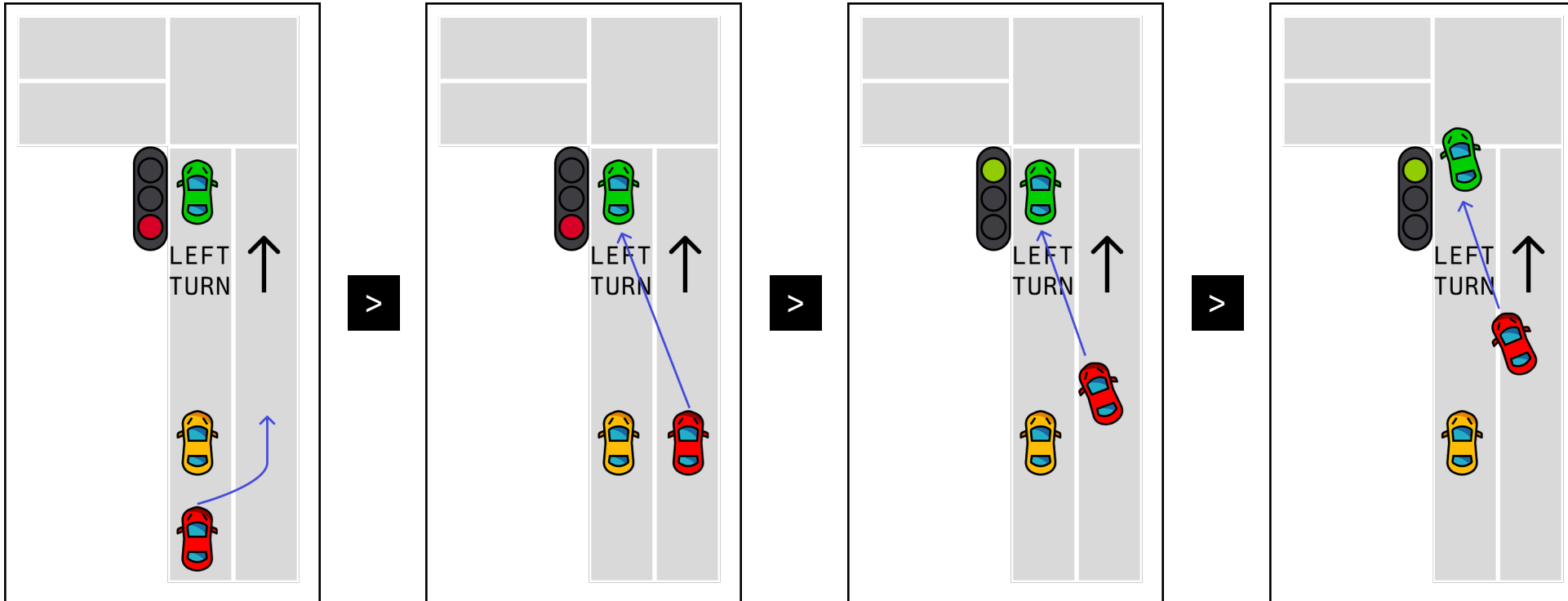
Failing Test Executions (4/20)

AV Cuts off another vehicle

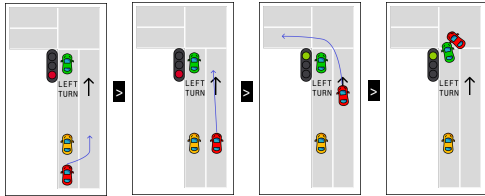


Passing Test Executions (16/20)

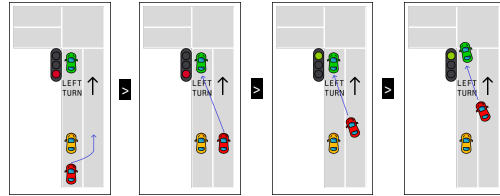
AV Returns to its original lane



Failing Test Executions (4/20) AV Cuts off another vehicle



Passing Test Executions (16/20) AV Returns to its original lane



Extended Study

AV-FUZZER

BehAVExplor

Decictor

DeepCollision

DoppelTest

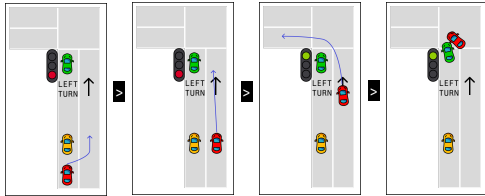
DriveFuzz

SAMOTA

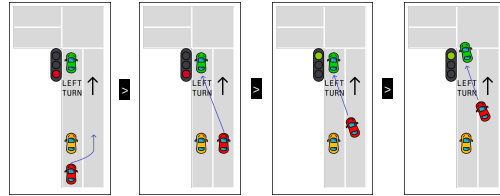
scenoRITA

6158 Scenario Tests

Failing Test Executions (4/20) AV Cuts off another vehicle



Passing Test Executions (16/20) AV Returns to its original lane



Extended Study

AV-FUZZER

BehAVExplor

Decictor

DeepCollision

DoppelTest

DriveFuzz

SAMOTA

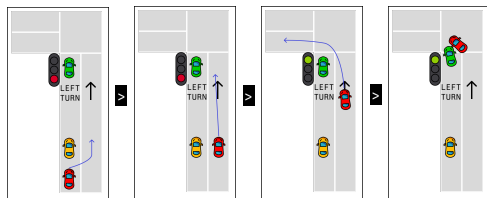
scenoRITA

6158 Scenario Tests

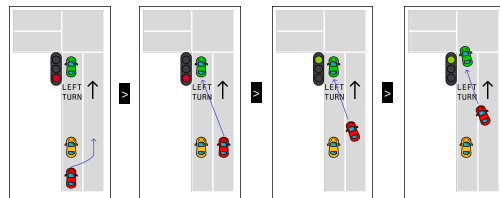


658 Collisions

Failing Test Executions (4/20) AV Cuts off another vehicle



Passing Test Executions (16/20) AV Returns to its original lane



Extended Study

AV-FUZZER

BehAVExplor

Decictor

DeepCollision

DoppelTest

DriveFuzz

SAMOTA

scenoRITA

6158 Scenario Tests



658 Collisions

187 Non-Deterministic Collisions
(28.41%)

Where does non-determinism
come from?



Parameter Coverage for Testing of Autonomous Driving Systems under Uncertainty

THOMAS LAURENT, Lero & University College Dublin, Ireland
STEFAN KLIKOVITS, PAOLO ARCAINI, and FUYUKI ISHIKAWA, National Institute of Informatics, Japan
ANTHONY VENTRESQUE, Lero & University College Dublin, Ireland

Autonomous Driving Systems (ADSs) are promising, but must show they are secure and trustworthy before adoption. Simulation-based testing is a widely adopted approach, where the ADS is run in a simulated environment over specific scenarios. Coverage criteria specify what needs to be covered to consider the ADS sufficiently tested. However, existing criteria do not guarantee to exercise the different decisions that the ADS can make, which is essential to assess its correctness. ADSs usually compute their decisions using parameterised rule-based systems and cost functions, such as cost components or decision thresholds. In this article, we argue that the parameters characterise the decision process, as their values affect the ADS's final decisions. Therefore, we propose parameter coverage, a criterion requiring to cover the ADS's parameters. A scenario covers a parameter if changing its value leads to different simulation results, meaning it is relevant for the driving decisions made in the scenario. Since ADS simulators are slightly uncertain, we employ statistical methods to assess multiple simulation runs for execution difference and coverage. Experiments using the Autonomoose ADS show that the criterion discriminates between different scenarios and that the cost of computing coverage can be managed with suitable heuristics.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**;

Additional Key Words and Phrases: Software testing, autonomous driving, coverage criteria, mutation analysis

ACM Reference format:

Thomas Laurent, Stefan Klikovits, Paolo Arcaini, Fuyuki Ishikawa, and Anthony Ventresque. 2023. Parameter Coverage for Testing of Autonomous Driving Systems under Uncertainty. *ACM Trans. Softw. Eng. Methodol.* 32, 3, Article 58 (April 2023), 31 pages.
<https://doi.org/10.1145/3550270>

This work was supported, in part, by Science Foundation Ireland grants 20/FFP-P/8818, and 13/RC/2094_P2 and co-funded under the European Regional Development Fund through the Southern & Eastern Regional Operational Programme to Lero - the Science Foundation Ireland Research Centre for Software (www.lero.ie). Stefan Klikovits, Paolo Arcaini, and Fuyuki Ishikawa are supported by the ERATO HASUO Metamathematics for Systems Design Project JPMJER1603; Funding Reference number 10.13039/501100009024 Japan Science and Technology Agency (JST). Stefan Klikovits is supported by Japan Society for the Promotion of Science (JSPS) Grant-in-Aid for Research Activity Start-up No 20K23334. Paolo Arcaini and Fuyuki Ishikawa are also supported by Engineerable AI Techniques for Practical Applications of High-Quality Machine Learning-based Systems Project (Grant Number JPMJM20B8), JST-Mirai.

Authors' addresses: T. Laurent and A. Ventresque, University College Dublin, School of Computer Science, Belfield, Dublin 4, Ireland; emails: thomas.laurent@ucdconnect.ie, anthony.ventresque@ucd.ie; S. Klikovits, P. Arcaini, and F. Ishikawa, National Institute of Informatics, 2-1-2 Hitotsubashi, Chiyoda-ku, Tokyo 101-8430, Japan; emails: sklikovits@nii.ac.jp, araini@nii.ac.jp, f-ishikawa@nii.ac.jp.



This work is licensed under a Creative Commons Attribution International 4.0 License.

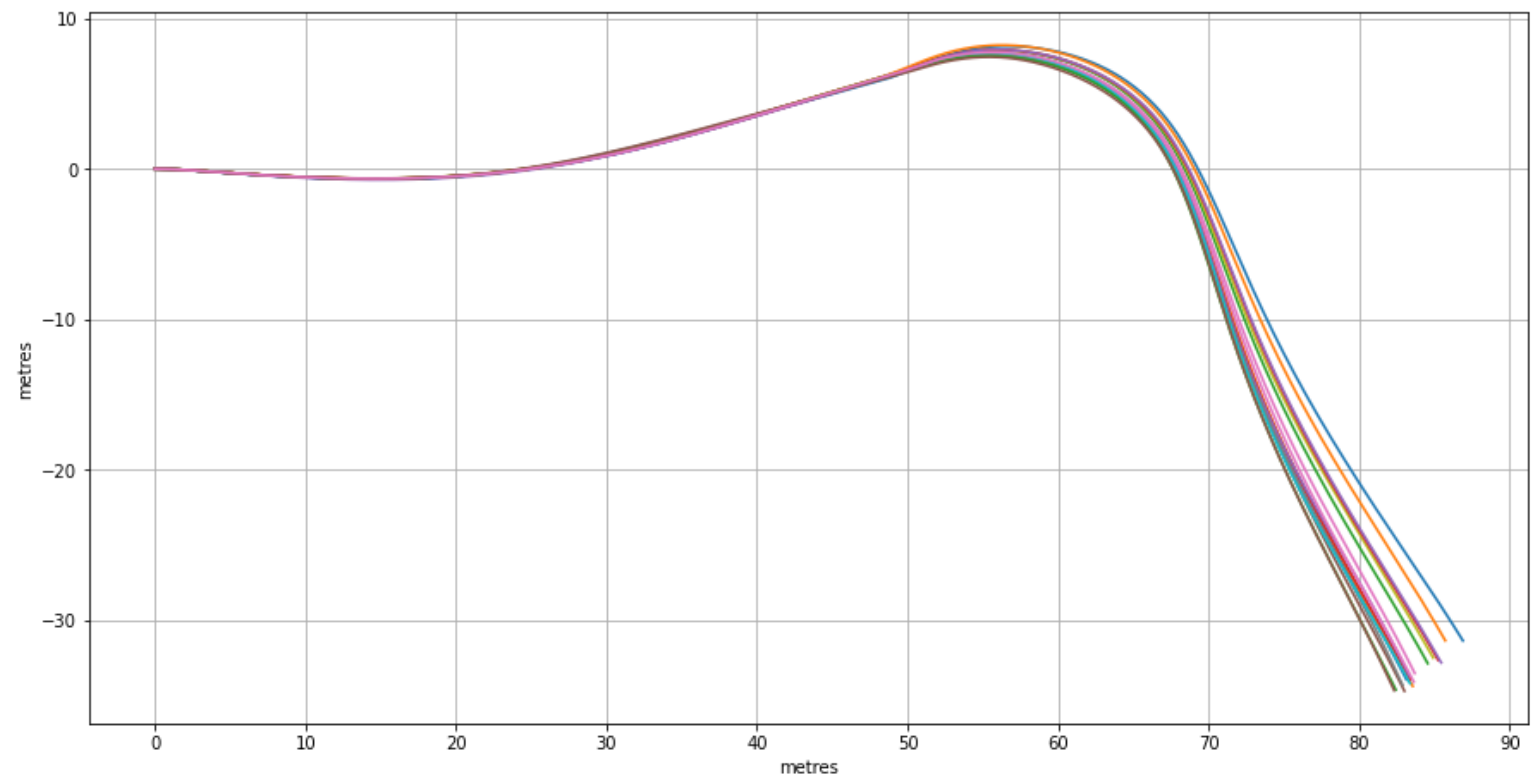
© 2023 Copyright held by the owner/author(s).
1049-331X/2023/04-ART58 \$15.00
<https://doi.org/10.1145/3550270>

ACM Transactions on Software Engineering and Methodology, Vol. 32, No. 3, Article 58. Pub. date: April 2023.

58

Laurent et. al

“Repeated executions of the exact same scenario and parameterization lead to diverging paths.”



ADS is non-deterministic at system level.

// Preliminary Studies

apollo ApolloAuto/apollo | ★ 26.5k

ApolloAuto / apollo

Code

Issues 917

Pull requests 104

Agents

Discussions

Actions

Wiki

Security and quality

Insights

apollo

Public

Edit Pins

Watch 1096

Fork 9.9k

Starred 26.5k

master

Go to file

Code

README

Code of conduct

Contributing

Apache-2.0 license

Baidu 百度 | apollo

[Build Status](#) [Simulation Status](#)

We choose to go to the moon in this decade and do the other things,
not because they are easy, but because they are hard.

— John F. Kennedy, 1962

Welcome to Apollo's GitHub page!

Apollo is a high performance, flexible architecture which accelerates the development, testing, and deployment of Autonomous Vehicles.

For business and partnership, please visit [our website](#).

Table of Contents

1. Introduction

2. Prerequisites

3. Individual Versions

4. Architecture

About

An open autonomous driving platform

apollo

self-driving-car

autonomous-driving

autonomous-vehicles

autonomy

Readme

Apache-2.0 license

Code of conduct

Contributing

Activity

Custom properties

26.5k stars

1.1k watching

9.9k forks

Report repository

Releases 12

apollo-v9.0.0

Latest

on Dec 18, 2023

+ 11 releases

Deployments 500+

github-pages

2 months ago

+ more deployments

autowarefoundation/autoware | ★ 11.3k

autowarefoundation / autoware

Code

Issues 71

Pull requests 11

Agents

Discussions

Actions

Security and quality

Insights

autoware

Public

Edit Pins

Watch 435

Fork 3.6k

Starred 11.3k

main

Go to file

Code

README

Code of conduct

Contributing

Apache-2.0 license

Autoware - the world's leading open-source software project for autonomous driving

Autoware Universe Contributors 250

Autoware Contributors 82

Autoware Universe Commit Activity 145/month

Autoware Commit Activity 39/month

License Apache-2.0

health-check passing

Coverage 19%

About

Autoware - the world's leading open-source software project for autonomous driving

[www.autoware.org/](#)

ros

autonomous-driving

autonomous-vehicles

ros2

autoware

Readme

Apache-2.0 license

Code of conduct

Contributing

Activity

Custom properties

11.3k stars

435 watching

3.6k forks

Report repository

Releases 9

1.7.1

Latest

on Feb 18

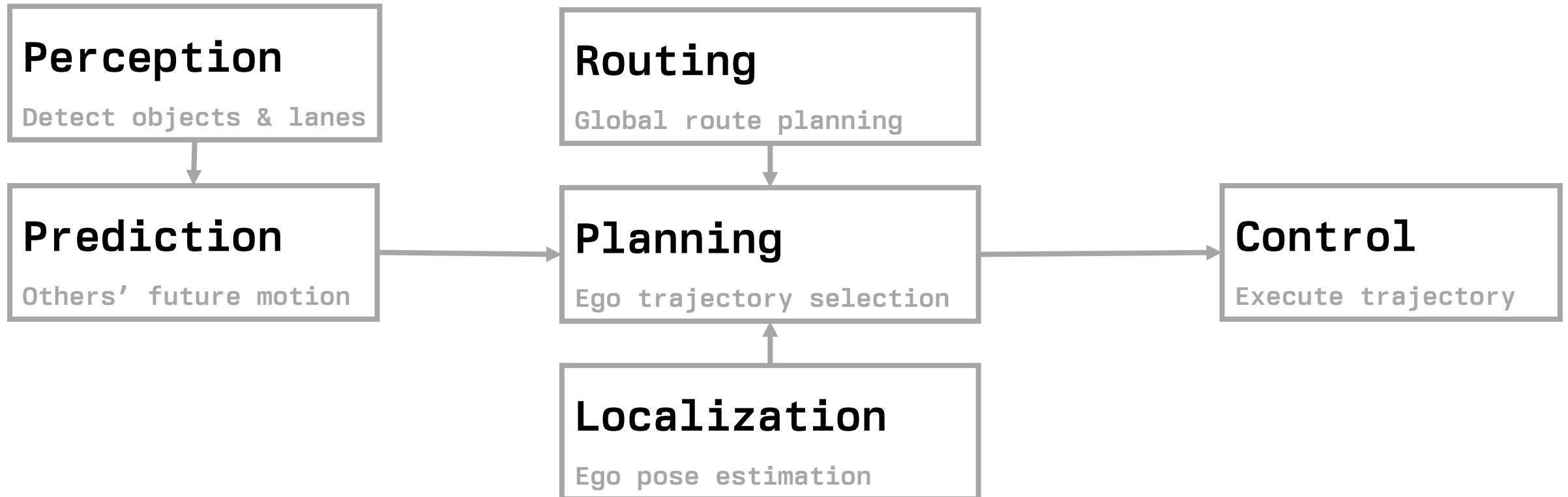
+ 8 releases

Packages 3

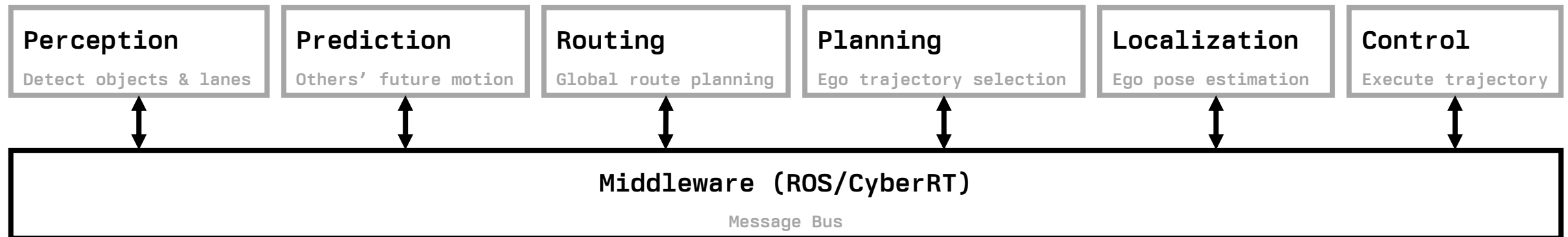
autoware

autoware-base

Multi-module Architecture



Multi-module Publish-Subscribe Architecture

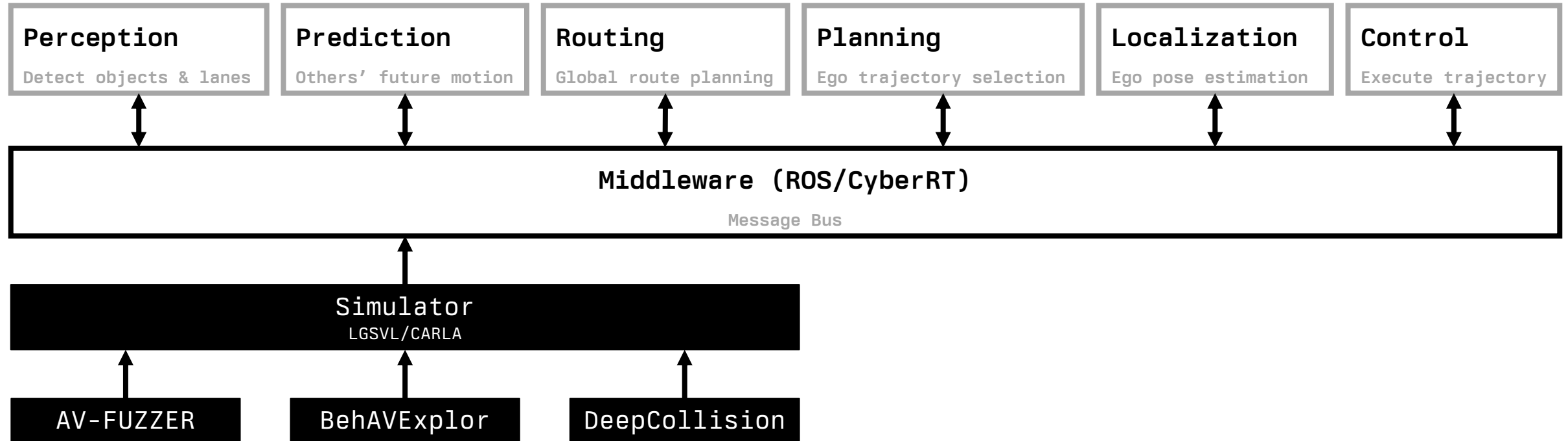


Communication between modules only implicitly exists through the message bus!

apollo ApolloAuto/apollo | ★ 26.5k

 autowarefoundation/autoware | ★ 11.3k

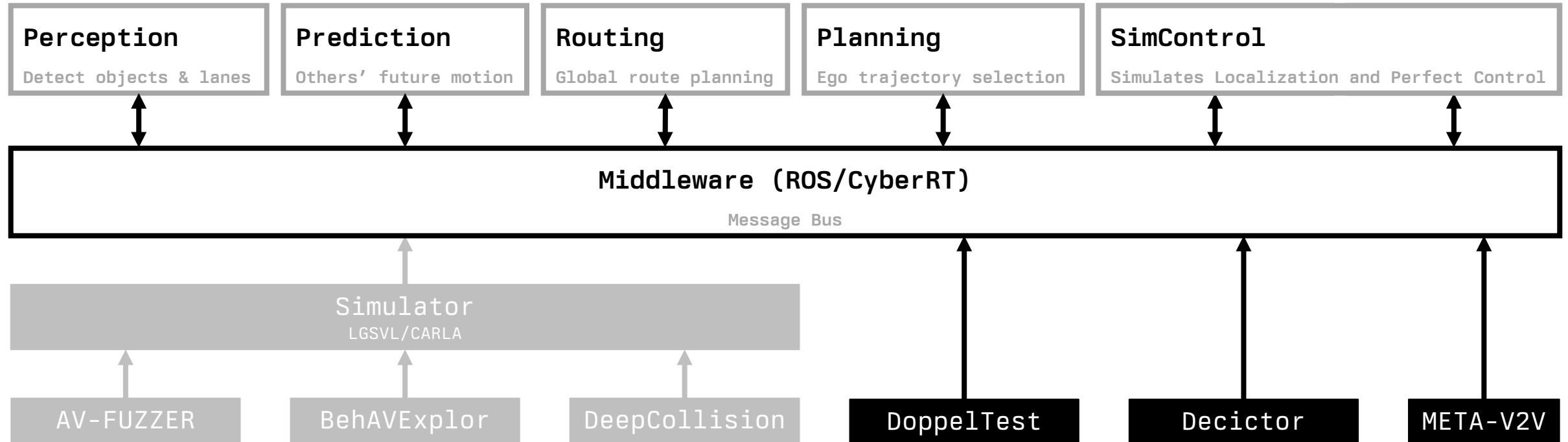
Multi-module Publish-Subscribe Architecture



apollo ApolloAuto/apollo | ★ 26.5k

 autowarefoundation/autoware | ★ 11.3k

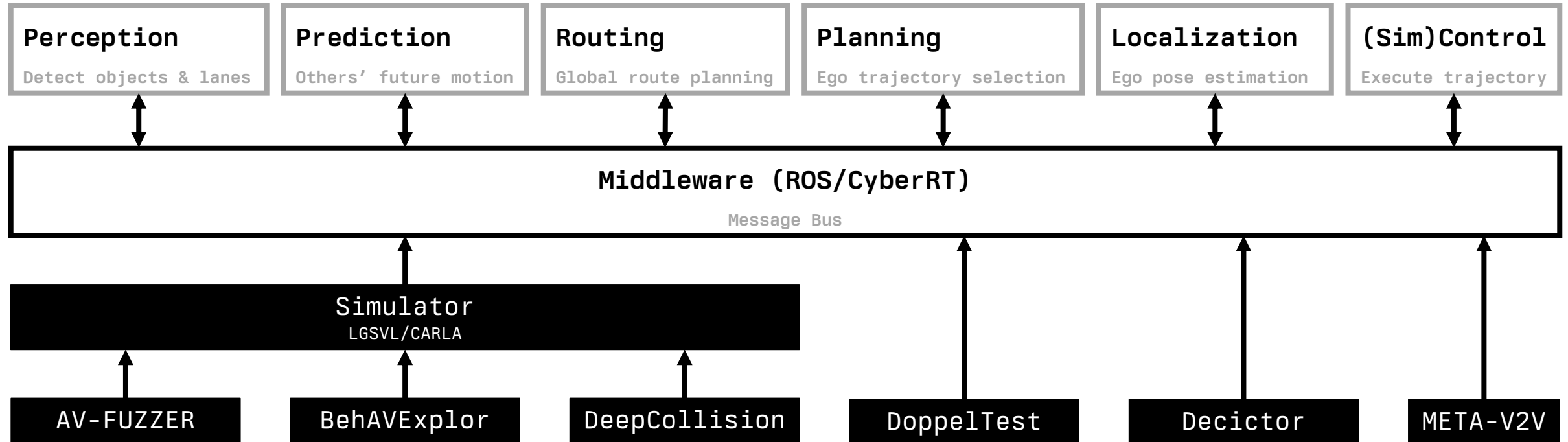
Multi-module Publish-Subscribe Architecture



apollo ApolloAuto/apollo | ★ 26.5k

 autowarefoundation/autoware | ★ 11.3k

Multi-module Publish-Subscribe Architecture



No explicit control over when and what inputs are processed by ADS modules.

apollo ApolloAuto/apollo | ★ 26.5k



autowarefoundation/autoware | ★ 11.3k

Planning

Ego trajectory selection * Decides the AV's trajectory (behavior)

apollo ApolloAuto/apollo | ★ 26.5k



autwarefoundation/autware | ★ 11.3k

Planning

Ego trajectory selection * Decides the AV's trajectory (behavior)

From: Developer Confirmation

- Implementation of the planning module is deterministic.

From: Experiments

Planning module's output varies during repeated simulations of the same scenario.

Can a testing method maintain
determinism of ADS planning?

Frame-based Testing

Deterministic Frame-based Iesting

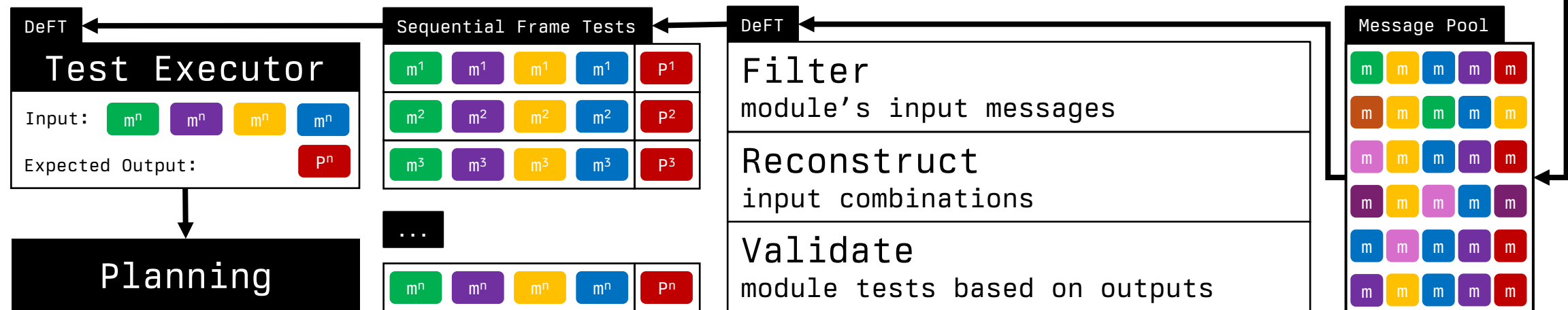
Deterministic when the module under test is deterministic.

// Deterministic Frame-based Iesting

Scenario-based Testing



Frame-based Testing



// Deterministic Frame-based Iesting

Filter

What are the messages that are relevant to the planning module?

Reconstruct

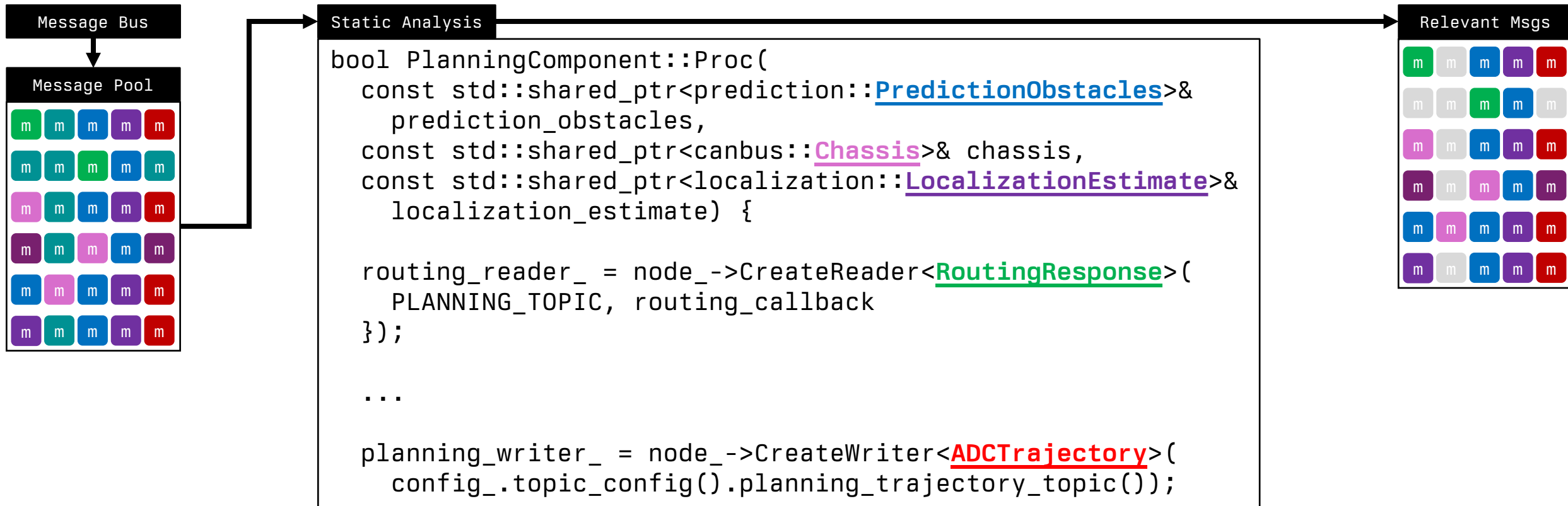
Validate

M : all messages on the message bus.

S : set of channels the planning module subscribes to.

P : set of channels the planning module publishes to.

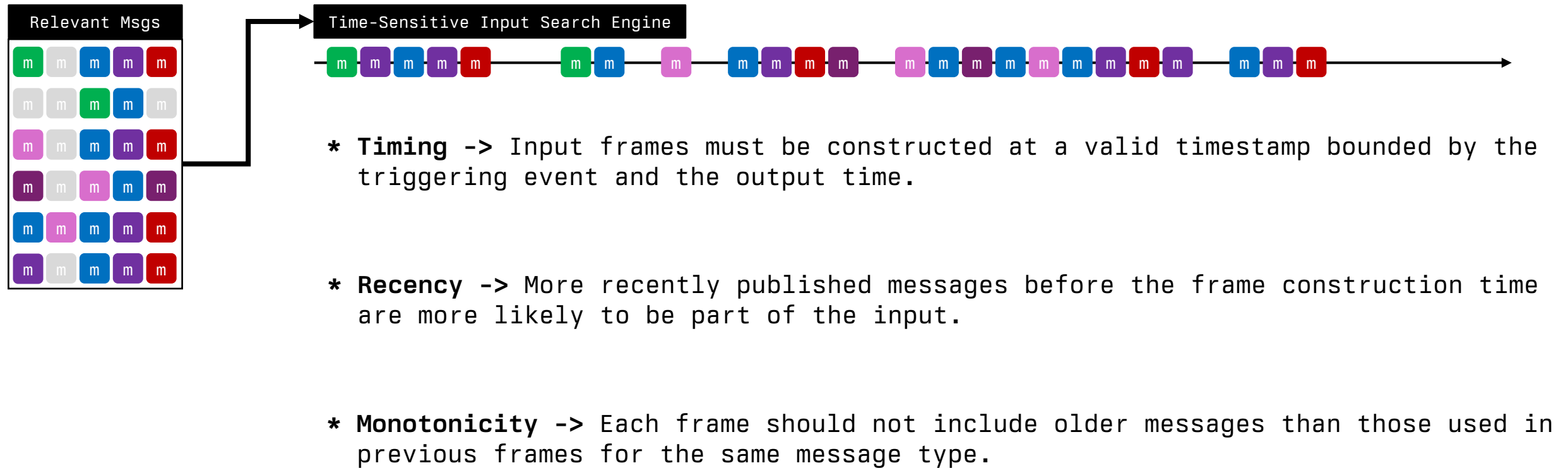
$$R = \{m \in M \mid \text{channel}(m) \in \text{SUP}\}$$



// Deterministic Frame-based Iesting

Filter	Reconstruct When and what combination did the planning module process?	Validate
--------	---	----------

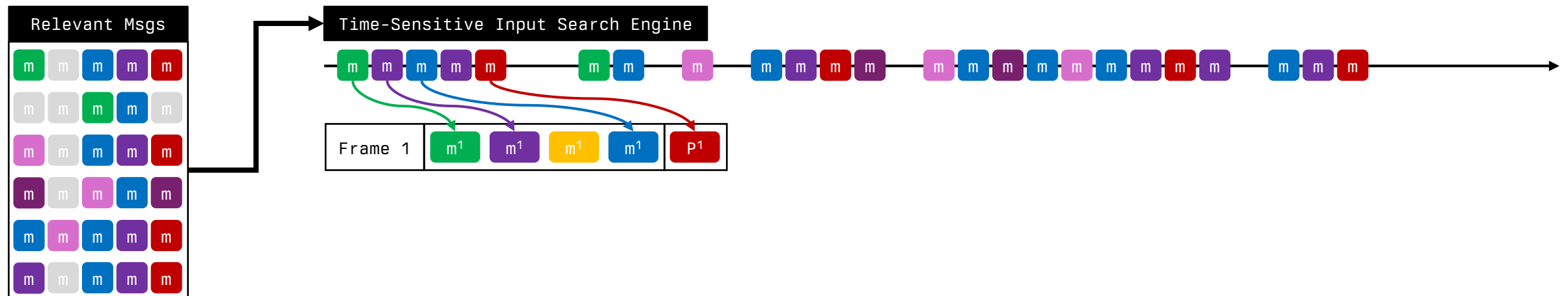
During system-level testing, Planning module is triggered by **events** to aggregate messages from different channels to construct input frame $F = \langle m_{\text{routing}}, m_{\text{localization}}, m_{\text{traffic_light}}, m_{\text{prediction}} \rangle$ at time t_F to produce m_{planning} .



// Deterministic Frame-based Iesting

Filter	Reconstruct When and what combination did the planning module process?	Validate
--------	---	----------

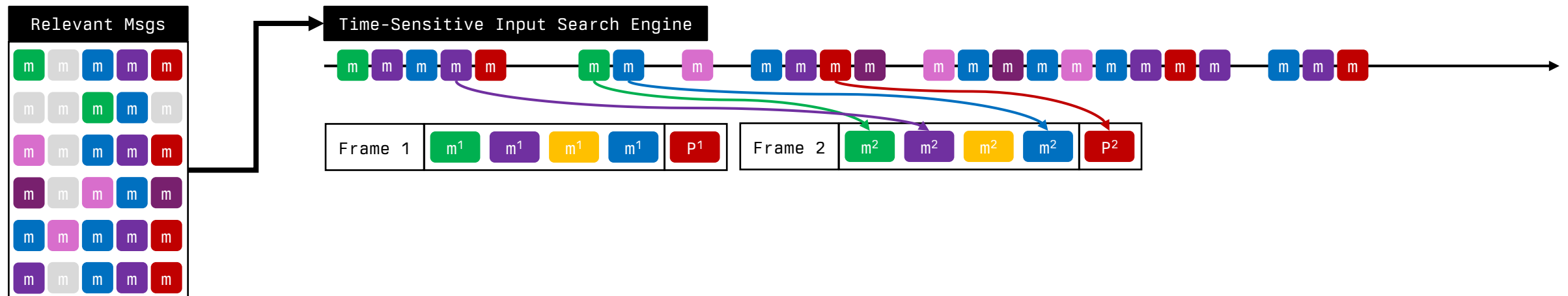
During system-level testing, Planning module is triggered by **events** to aggregate messages from different channels to construct input frame $F = \langle m_{\text{routing}}, m_{\text{localization}}, m_{\text{traffic_light}}, m_{\text{prediction}} \rangle$ at time t_F to produce m_{planning} .



// Deterministic Frame-based Iesting

Filter	Reconstruct When and what combination did the planning module process?	Validate
--------	---	----------

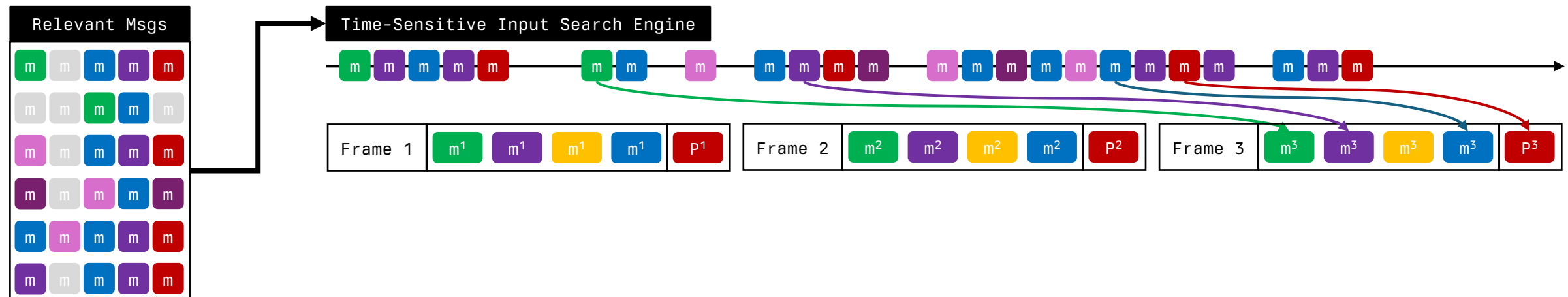
During system-level testing, Planning module is triggered by **events** to aggregate messages from different channels to construct input frame $F = \langle m_{\text{routing}}, m_{\text{localization}}, m_{\text{traffic_light}}, m_{\text{prediction}} \rangle$ at time t_F to produce m_{planning} .



// Deterministic Frame-based Iesting

Filter	Reconstruct When and what combination did the planning module process?	Validate
--------	---	----------

During system-level testing, Planning module is triggered by **events** to aggregate messages from different channels to construct input frame $F = \langle m_{\text{routing}}, m_{\text{localization}}, m_{\text{traffic_light}}, m_{\text{prediction}} \rangle$ at time t_F to produce m_{planning} .

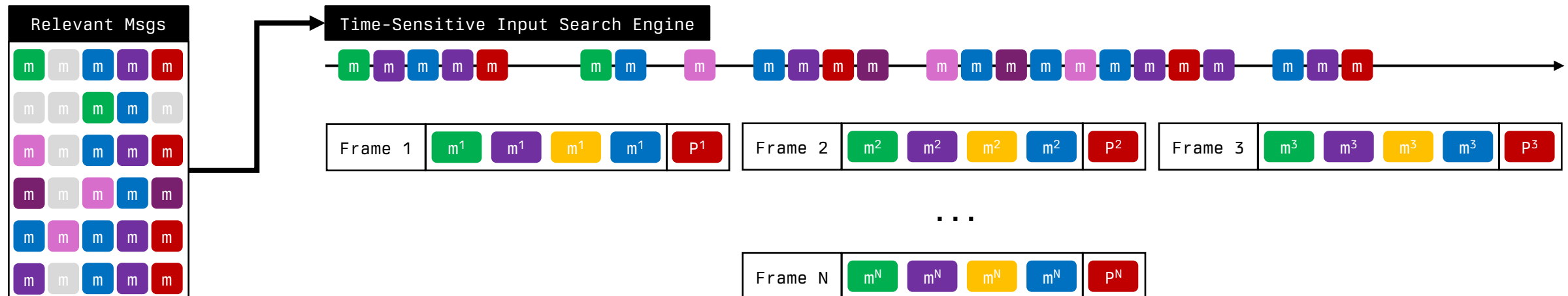


// Deterministic Frame-based Iesting

Filter	Reconstruct When and what combination did the planning module process?	Validate
--------	---	----------

During system-level testing, Planning module is triggered by **events** to aggregate messages from different channels to construct input frame $F = \langle m_{\text{routing}}, m_{\text{localization}}, m_{\text{traffic_light}}, m_{\text{prediction}} \rangle$ at time t_F to produce m_{planning} .

Each planning module execution -> A single module-level frame-based test case.



// Deterministic Frame-based Iesting

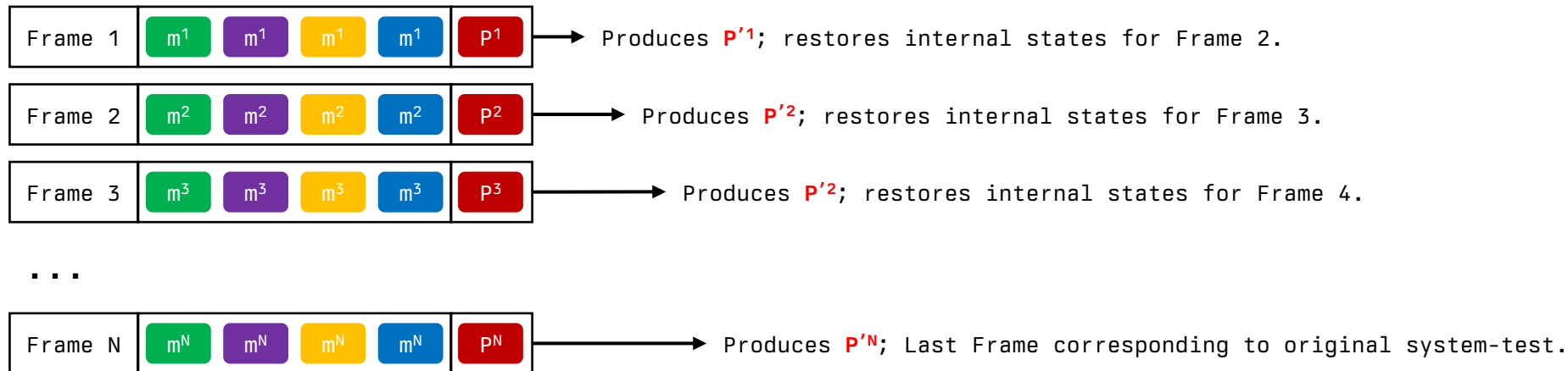
Filter	Reconstruct When and what combination did the planning module process?	Validate
--------	---	----------

During system-level testing, Planning module is triggered by **events** to aggregate messages from different channels to construct input frame $F = \langle m_{\text{routing}}, m_{\text{localization}}, m_{\text{traffic_light}}, m_{\text{prediction}} \rangle$ at time t_f to produce m_{planning} .

Each planning module execution -> A single module-level frame-based test case.

Module tests are executed **sequentially** s.t. executing frames 1 through $n-1$ restores the internal states of the planning module for the n^{th} frame.

Sequential Frame Tests

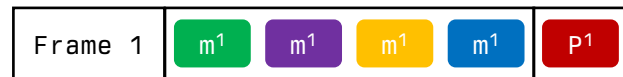


// Deterministic Frame-based Iesting

Filter	Reconstruct	Validate
--------	-------------	----------

Does processing the reconstructed frame reproduce the expected output?

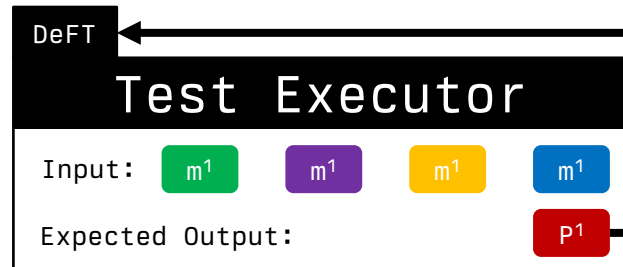
Each **Planning output (P)** consists of a list of **Path Points (p)** representing the it plans to reach position $\langle x, y \rangle$ at time t .



Trajectory Validator (Output Validation)

$$Eu(P, P') = \sqrt{\sum_{i=1}^{|P|} ((x_i^P - x_i^{P'})^2 + (y_i^P - y_i^{P'})^2)}$$

- + Validate correctness of the input frame by comparing planning module's output.
- + Uses Euclidean distance to calculate overall geometric difference across the entire planning trajectories.
- + Smaller distance = more accurate input frame construction.



Planning

Actual Output:

p'¹

RQ1 Planning Reproducibility

To what extent do scenario tests and module tests reproduce planning-module executions?

RQ2 Failure Reproducibility

To what extent can module tests reproduce failures detected by test generation approaches?

RQ3 Efficiency

What is the run-time efficiency of DeFT and the produced module tests?

Experiment Setup

apollo ApolloAuto/apollo | ★ 26.5k

8 Apollo Test Generators



ICSE 23, 25; TSE 23

DeFT-Apollo

- * Uses Apollo planning module' output to reconstruct inputs.
- * Converts each scenario test into a module-level test suite.

System Level Scenario Test Benchmark

Initial Test Suite

Scenario 1
...
Scenario N

Reruns of system-level scenario-based tests

S ¹ R ¹	S ¹ R ²	S ¹ R ³	S ¹ R ⁴	S ¹ R ⁵	S ¹ R ⁶	S ¹ R ⁷	S ¹ R ⁸	S ¹ R ⁹
...								
S ^N R ¹	S ^N R ²	S ^N R ³	S ^N R ⁴	S ^N R ⁵	S ^N R ⁶	S ^N R ⁷	S ^N R ⁸	S ^N R ⁹

Module Test Suites

MTS 1
...
MTS N

Reruns of module-level frame-based tests

M ¹ R ¹	M ¹ R ²	M ¹ R ³	M ¹ R ⁴	M ¹ R ⁵	M ¹ R ⁶	M ¹ R ⁷	M ¹ R ⁸	M ¹ R ⁹
...								
M ^N R ¹	M ^N R ²	M ^N R ³	M ^N R ⁴	M ^N R ⁵	M ^N R ⁶	M ^N R ⁷	M ^N R ⁸	M ^N R ⁹

RQ1 Planning Reproducibility

To what extent do scenario tests and module tests reproduce planning-module executions?

RQ2

RQ3

Planning reproducibility is meaningful with respect to failure reproducibility.

Collision Scenario

S¹R¹

S¹R²

S¹R³

S¹R⁴

S¹R⁵

S¹R⁶

S¹R⁷

S¹R⁸

S¹R⁹

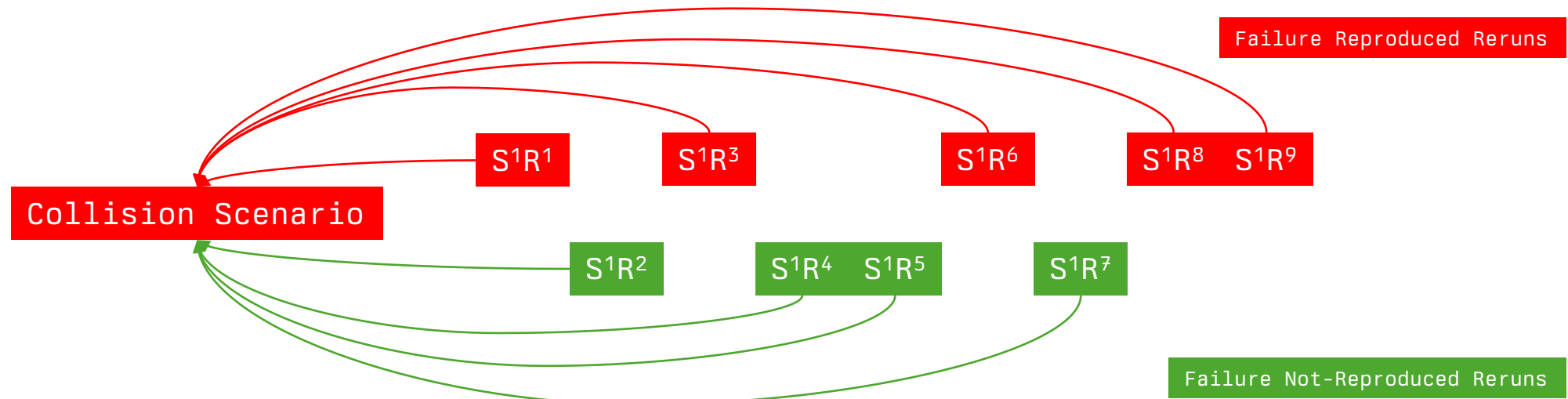
RQ1 Planning Reproducibility

To what extent do scenario tests and module tests reproduce planning-module executions?

RQ2

RQ3

Planning reproducibility is meaningful with respect to failure reproducibility.



Hypothesis 0:

Failures are not reproduced when ADS planning trajectories differ.

RQ1 Planning Reproducibility To what extent do scenario tests and module tests reproduce planning-module executions?	RQ2	RQ3
---	-----	-----

Measure	Failure Reproduced Edit Distances (m)	Failure Not-Reproduced Edit Distances (m)
1 st Quartile	0.10	1.62
2 nd Quartile	0.23	17.15
3 rd Quartile	6.99	308.71
min	0.0	0.0
mean	29.56	139.51
max	2320.81	1453.06

Table 1. Summary statistics of planning trajectory edit distances across reruns of a system-level scenario test.

Failures revealed by system-level scenario tests tend to be reproduced when planning trajectories remain similar to the original simulation.

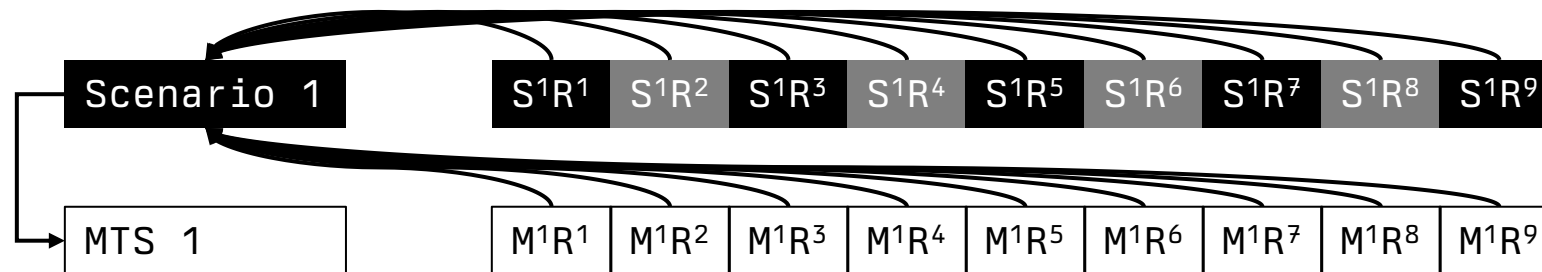
RQ1 Planning Reproducibility

To what extent do scenario tests and module tests reproduce planning-module executions?

RQ2

RQ3

Compare planning trajectories from reruns of the scenario test or module tests with the initial simulation.



Hypothesis 1:

Module tests resemble the executions occurred during system-level scenario testing; therefore, running module tests should replicate the planning trajectories observed in those system-level scenarios.

RQ1 Planning Reproducibility To what extent do scenario tests and module tests reproduce planning-module executions?	RQ2	RQ3
---	-----	-----

Threshold (m)	Module Test Rerun	System-Test Rerun
0.0	224,888	1,637
0.1	228,956	42,335
0.23	228,975	95,235
1.62	229,108	127,258
6.99	229,915	154,467
17.15	230,637	173,282
29.56	230,852	184,047

Table 2. Number of planning trajectories reproduced by DeFT module tests compared to rerunning scenario tests under different thresholds.

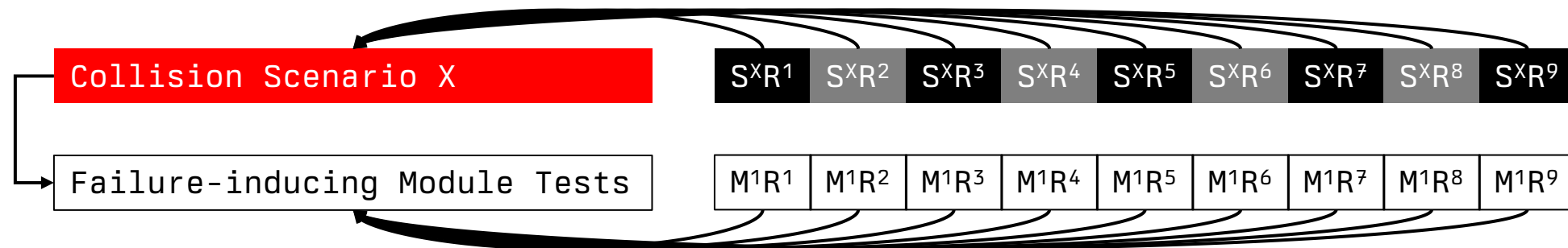
DeFT module tests reproduces more planning trajectories across different thresholds.

RQ2 Failure Reproducibility

To what extent can module tests reproduce failures detected by test generation approaches?

RQ3

Compare test oracle outputs from reruns of the scenario or module tests with those from the initial simulation/execution.



Hypothesis 2:

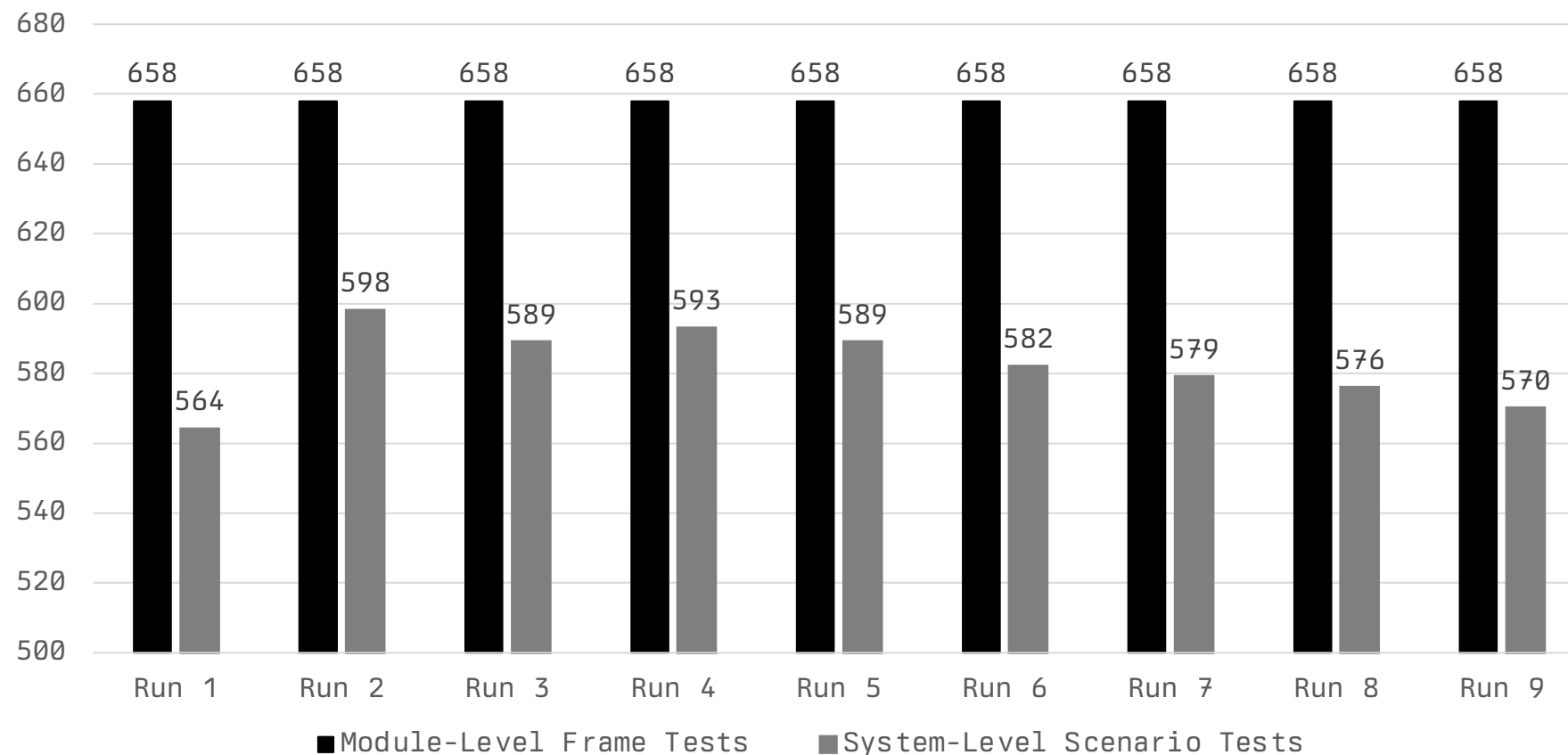
System tests are non-deterministic, so collisions observed in the initial simulation may not occur again.

In contrast, module tests are deterministic, meaning repeated executions should consistently yield the same outputs.

RQ2 Failure Reproducibility

To what extent can module tests reproduce failures detected by test generation approaches?

RQ3

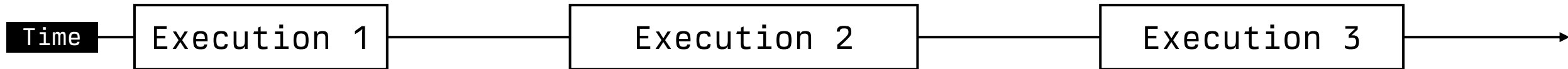


Rerunning simple scenario tests does not reliably reproduce all violations, whereas the DeFT module test provides deterministic results.

RQ3 Efficiency

What is the run-time efficiency of DeFT and the produced module tests?

System Testing - Idle time occurs in each module execution cycle.



Module Testing - Shorter frame input read intervals occur between module executions.



Hypothesis 3:

Since a smaller portion of the ADS is exercised, and planning module is directly invoked, module-level testing is expected to reduce execution time.

RQ3 Efficiency

What is the run-time efficiency of DeFT and the produced module tests?

Approach	Deterministic Frame-Based Testing			System Test (s)	Reduction
	Extraction	Execution	Total		
AV-FUZZER	21.22	54.23	75.34	86.33	12.73%
BehAVExplor	44.54	86.03	130.57	138.43	5.67%
Decictor	22.50	54.55	77.05	84.85	9.19%
DeepCollision	13.84	39.23	53.07	60.90	12.86%
DoppelTest	12.77	18.69	31.46	35.77	12.04%
DriveFuzz	22.11	57.17	79.27	88.47	10.40%
scenoRITA	14.70	21.88	36.57	34.79	-5.14%

Table 3. Run-time cost of running DeFT compared to system-level scenario tests.

Module-level testing offers a more efficient alternative to system-level testing in terms of runtime cost.

apollo

Docker Version

Co-Driver

Mute

Mkz Standard Debug

-- vehicle --

San Mateo

Tasks

Module Controller

Layer Menu

Route Editing

Data Recorder

15 km/h

Brake 0%

Accelerator 0%

0%

GREEN AUTO

Gear D

461010

(3.7m, 1.6m/s)

Quick Start

Setup

Reset All

Start Auto

Others

Reset Backend Data

Dump Message

Lock Task Panel

PNC Monitor

Sim Control

Camera Sensor

Module Delay

Chassis 00:00.012

Control -

Localization 00:00.014

PerceptionObstacle 00:00.112

Planning 00:00.137

Prediction 00:00.142

TrafficLight 00:00.142

Console

You haven't selected a vehicle yet!

You haven't selected a vehicle yet!

You haven't selected a vehicle yet!

You haven't selected a vehicle yet!

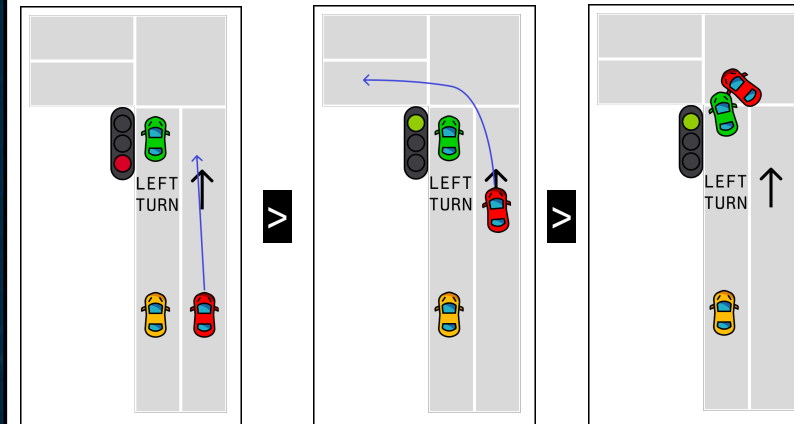
You haven't selected a vehicle yet!

You haven't selected a vehicle yet!

You haven't selected a vehicle yet!

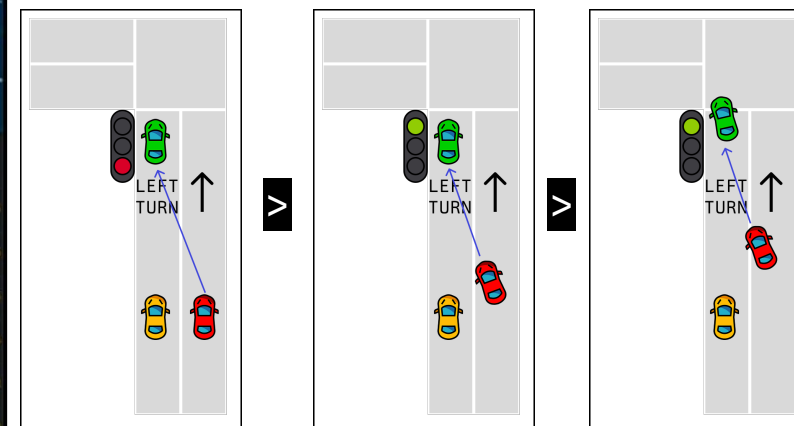
Failing Test Executions (4/20)

AV Cuts off another vehicle



Passing Test Executions (16/20)

AV Returns to its original lane



In the scenario-based test world ...

Test Specification

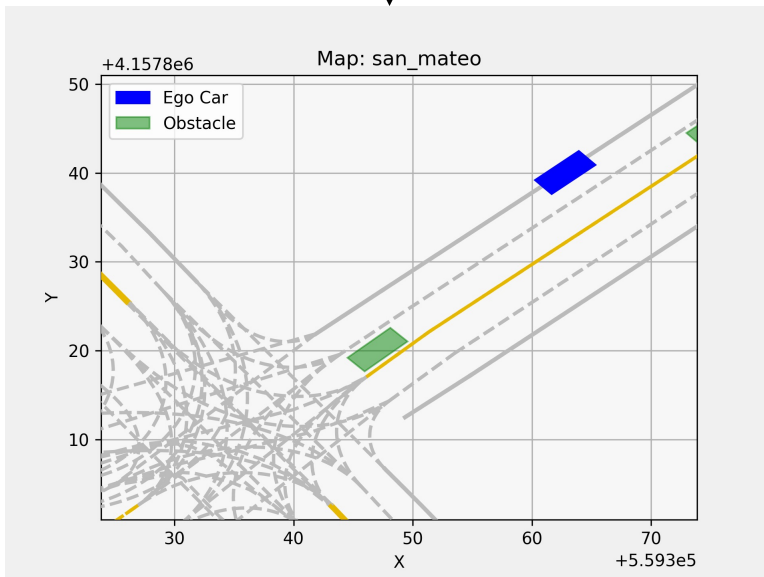
Simulation

Hope for reproduction ☹

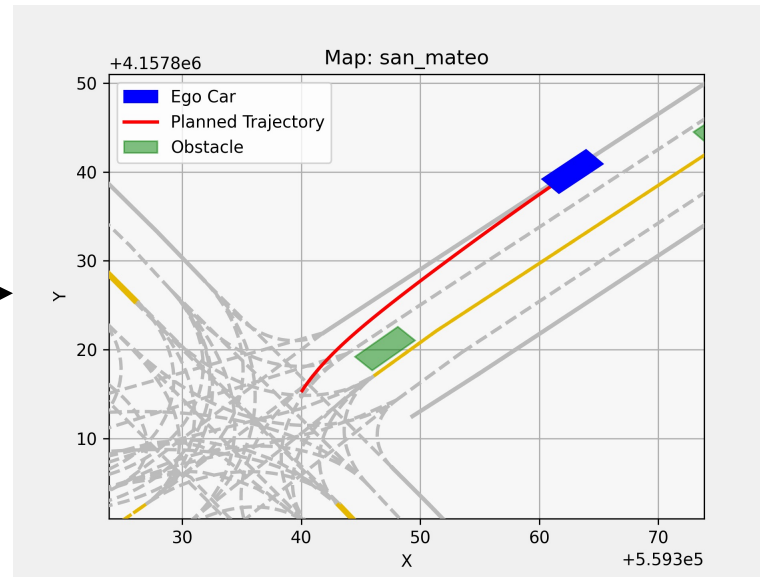
In the DeFT world ...

Sequential Module-Level Frame-Based Tests

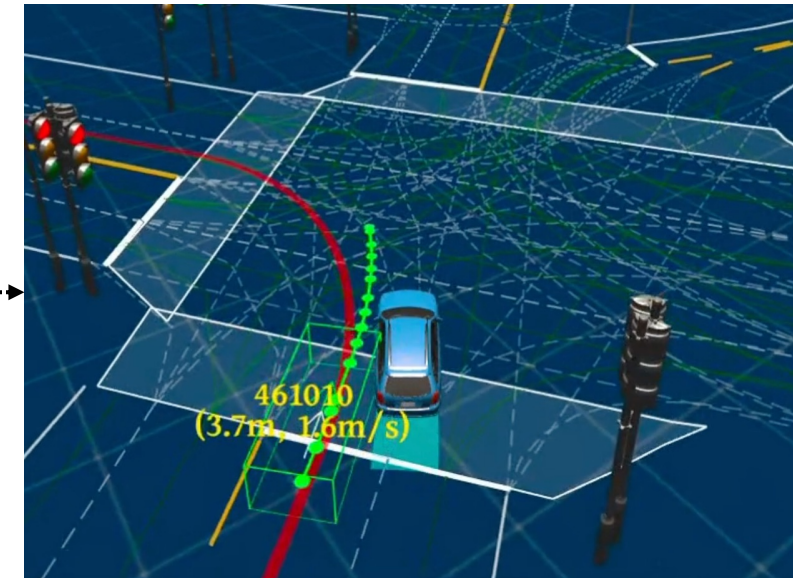
Sets of Inputs and Expected Outputs



Visualization of the frame input.



Visualization of the faulty **planning output**.
DETERMINISTIC!



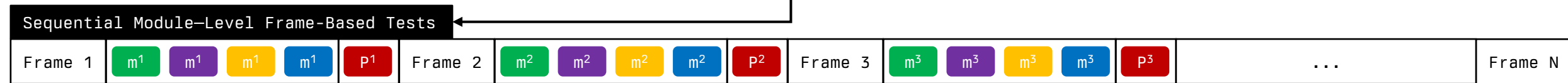
Known collision from the system-test.

// Conclusion

Scenario-based Testing ☹️



Deterministic Frame-based Testing 😊



Realistic

Module-tests can accurately reproduce planning behaviors.

~~0.005%~~ -> 97.16%
Planning Reproduction

Deterministic

Module-tests yield predictable results when the module-under-test is deterministic.

~~71.58%~~ -> 100%
Failure Reproduction

Efficient

Module-tests are faster to execute comparing to simulation.

5.67% - 12.86%
Time Saving

Key Points // DeFT turns flaky system-level tests into deterministic, reproducible module-level tests.

- * **Non-determinism** of system-level testing **arises from real-time communication** between modules.
- * DeFT **extract module-level tests** that **deterministically reproduces** the planning module's behavior.
- * The planning module's outputs can be used to both **infer and validate** its **reconstructed inputs**.

Future Work

- * Differential Testing // measure output change
- * Mutation Testing // mutant killing capability



Yuqi Huai <yhuai@uci.edu>
University of California, Irvine

